

AsymptoticAnalysis

Numerical contracts, Mathics semantics,
and validation infrastructure

A differential audit of the Asymptotic repository

Repository	VladimirReshetnikov/Asymptotic
Reviewed revision	7d1bc832895cc90a9b2a978b7b7684acab908bd2
Commit timestamp	September 10, 2026, 02:10:22 UTC September 9, 2026, 19:10:22 America/Los_Angeles
Compatibility target	Mathics3 10.0.1; scanner 10.0.1; SymPy 1.14.0; Python 3.11, as declared by the project
Review baseline	The maintained crosswalks for 27 existing reports and their 167 identified ledger entries

Evidence boundary. This review inspected pinned source and primary documentation. It executed **25 independent Python tests**, including exact-rational certificates and synthetic process/snapshot checks. It did **not** execute the package in Mathics or an official Wolfram kernel: those runtimes were not reachable in this session. Supplied Wolfram-language probes are explicitly unexecuted characterization programs.

Central conclusion. The repository has a meaningful role as a specialized, remainder-aware real asymptotic calculus. Its Mathics port is substantial but not complete. The highest-priority new concerns lie at the boundary between exact symbolic results and numerical evaluation, followed by internal semantic adapters and the integrity of portable validation. Prepared for Vladimir Reshetnikov.

The accompanying archive contains this article, source code, a novelty ledger, executed independent evidence, and fresh-kernel probe infrastructure.

Contents

1	Executive assessment	3
2	Scope, sources, and nonduplication	4
2.1	What was inspected and what was executed	4
2.2	How earlier reviews were used	5
2.3	A compatibility result needs four coordinates	5
3	Comparison with built-in Wolfram Language	5
3.1	Representation rather than a function-name contest	6
3.2	Native asymptotics is broader than Taylor expansion	6
3.3	Four levels of evidence about an inverse	7
3.4	Assessment of performance claims	8
4	Mathics compatibility: substantial work, incomplete equivalence	8
4.1	The implementation is a scoped semantic adapter	8
4.2	Capability should be stratified	9
4.3	What the repository's receipts establish	9
4.4	User-facing consequences	10
5	F01: the high-precision root route is not portable	10
5.1	The data-flow mismatch	10
5.2	Why current smoke tests are weak controls for this issue	11
5.3	Repair and acceptance criteria	11
6	F02: retained nonprincipal Lambert cores remain numerically hazardous	11
6.1	What is fixed and what still escapes	11
6.2	The mathematical branch is unambiguous	12
6.3	Do not overgeneralize the witness	12
6.4	Repair before evaluation, not after it	13
7	F03: default limits change mathematical meaning	13
8	F04: first-position grammar and avoidable traversal	13
8.1	An option is consumed as a default value	14
8.2	The first result is obtained by storing every result	14

9	Portable validation: three focused runner findings	14
9.1	F05: nonfinite timeouts pass the input guard	15
9.2	F06: boundary equality is not temporal source immutability	15
9.3	F07: diagnostics have no memory budget	16
10	A proved, executable fallback for a small Lambert inverse	16
10.1	A rational enclosure for the logarithm	16
10.2	A uniformly valid initial bracket	17
10.3	Bisection with exact sign decisions	17
10.4	Executed result and its precise scope	18
11	Three concrete compatibility improvements	18
11.1	E01: solve the rational affine neighborhood problem instead of sampling radii	18
11.2	E02: admit terminating hypergeometric series before convergence screening	19
11.3	E03: make late adapter rewrites assert their installation invariants	20
12	Development priorities and acceptance plan	20
12.1	Gate 1: numerical honesty before numerical breadth	20
12.2	Gate 2: helper contracts and bounded primitive work	20
12.3	Gate 3: acceptance records must describe the executed bytes	21
12.4	Gate 4: bounded extensions with independent oracles	21
13	Artifact guide and reproducibility	21
13.1	What the archive contains	21
13.2	Run the independent tests	22
13.3	Run package characterizations on a machine with a runtime	22
13.4	What was not delivered	23
14	Conclusion	23
A	Novelty and overlap ledger	23
B	Source locations and review coverage	24

1 Executive assessment

The most useful interpretation of this project is not “a replacement for **Series**.” It is a collection of specialized real asymptotic engines, coordinated by an object that retains a finite expression, a coordinate, an error description, assumptions, branch information, and reconstruction state. That combination is valuable when an expansion must subsequently be multiplied, inverted, composed, refined, or checked, rather than merely printed. The implementation already includes ordinary power-log jets, generalized inverse coefficients, logarithmic and exponential cores, finite Fourier coefficients in a logarithm, flat sectors, and dedicated Gamma/Barnes families. Native delegation now supplies a separate compatibility route rather than pretending every native result has the package’s analytic contract.[2, 8, 9]

The appropriate comparison with Wolfram Language is consequently two-level. Native **SeriesData** and **InverseSeries** supply efficient structured local series machinery; **Asymptotic** and **AsymptoticSolve** cover substantially broader asymptotic problems than a Taylor-only comparison would suggest. The repository adds particular representations, coefficient algorithms, retained real-branch information, and explicit error transport. Native support for a function is not identical to closure of the repository’s chosen representation, and a custom representation is not evidence that Wolfram cannot compute the same mathematical expansion.[35, 36, 37, 38]

The current Mathics work deserves credit. It addresses evaluation control, lazy defaults, held callable syntax, assumptions, exact series construction, numerical special-function hazards, empty-list behavior, and loading/reloading isolation. The documentation expressly states that complete compatibility remains a goal rather than an achieved result. Its recorded symbolic and exact-arithmetic examples must not be generalized into arbitrary-precision numerical equivalence.[9, 10, 11, 19]

This article identifies seven focused findings. F01 and F02 concern numerical capability boundaries; F03 and F04 concern helper semantics; F05–F07 concern the portable runner. Some are source-proved implementation defects, while the end-to-end reachability of others still requires the supplied probes. They are deliberately not bundled into an inflated count of seven experimentally reproduced package failures.

ID	Priority / scope	Finding	Evidence here
F01	High; numerical contract	Mathics’ root-finder path reduces the seed and subsequent arithmetic to machine precision; the package requests and describes high-precision evidence without checking achieved precision.	Source trace; public output unrun.
F02	High; retained exact cores	Nonprincipal ProductLog remains an active numerical dependency after the principal-branch workaround; a retained exact core reaches that dependency through ordinary substitution.	Source trace; independent numerical countermodel.
F03	Medium; internal helper	The Mathics Limit adapter maps its omitted direction to a left-sided limit rather than the current native two-sided default.	Source and documented semantics; explicit-direction consumer is a control.

ID	Priority / scope	Finding	Evidence here
F04	Medium; internal helper	FirstPosition confuses an option-only third argument with its default and constructs all matches before selecting the first.	Source and documented grammar; runtime probes supplied.
F05	Medium; runner input	The positive-timeout check admits NaN and positive infinity.	Exact predicate reproduced independently.
F06	Medium; evidence integrity	Pre/post file equality is labeled as unchanged throughout the run while package source remains live.	Executed edit/revert countermodel; Git-object snapshot prototype tested.
F07	Medium; process resources	Captured kernel output is unbounded and repeatedly serialized into the accumulated report.	Source analysis; bounded supervisor tested with synthetic children.

The supplied development artifacts go beyond recommendations. They provide a small exact certificate for the inverse of $-x \log x$, a rational affine eventual-neighborhood solver, a terminating hypergeometric coefficient generator, a bounded POSIX process supervisor, and a Git-object snapshot runner. These are reference implementations with explicit domains, not a claim that a replacement package has passed native acceptance.

2 Scope, sources, and nonduplication

2.1 What was inspected and what was executed

The repository revision is fixed at 7d1bc832895c; all repository links in this article resolve to that commit. The upstream Mathics implementation is read at its 10.0.1 release tag, matching the package’s declared target. Current official Wolfram documentation is used for language contracts. Public documentation can evolve; the pinned repository and explicit interpreter version therefore remain essential to interpreting a later reproduction.[1, 9, 29, 31]

Inspection concentrated on the public kernel entry, numerical inverse checks, Lambert and exact-core constructors, the relevant inverse-branch machinery, Fourier coefficient construction, the Mathics bootstrap and adapters, the portable test runner, its selected test definitions, and the compatibility/development registers. This is an architecture and targeted source audit, not a claim that every line of every special-function module or every historical article was read and executed.

An attempt to obtain an official Wolfram runtime through the available evaluator failed with a connection error. The local environment had neither Mathics nor a Wolfram kernel; external installation/download attempts were unavailable. Consequently, neither a package pass rate nor a package timing is reported from this session. Source predictions are not disguised as transcripts.

The executed evidence consists of 25 Python tests on Linux/Python 3.13.5. Exact arithmetic uses the standard-library `Fraction` type. Selected independent numerical cross-checks use `mpmath` 1.3.0 with elevated precision. No `SymPy` or `mpmath` numerical result is used as a premise of the rational certificate verifier. The transcript and machine-readable summary are in `evidence/independent-checks.txt` and `evidence/independent-checks.json`.

For clarity, the following evidence categories are kept separate throughout:

Source fact

A definition, dispatch condition, data-flow edge, or guard visible in the pinned source.

Mathematical deduction

A conclusion proved from an explicit formula or algorithm, independent of a particular evaluator.

Independent execution

A supplied Python implementation or countermodel run in this session, not execution of the repository.

Repository receipt

A test result reported by the repository, with its own baseline, source scope, and runtime.

Unexecuted characterization

A concrete program supplied to settle evaluator behavior or public reachability in a suitable fresh kernel.

2.2 How earlier reviews were used

The existing review collection is unusually extensive. The maintained register crosswalks the 123 wave-1/2 ledger entries; the wave-3 intake adds 44 entries from reports 19–27. These are 167 identified entries, not 167 distinct defects. The present audit used these complete maintained crosswalks, their descriptions of unnumbered proposals, the review catalog, and relevant original ledger material, including report 6. It did not rerun the old review programs or claim to inspect every page of all 27 articles.[25, 26, 27, 28]

This distinction matters for novelty. The present numerical finding is not a rediscovery that a huge source translation can lose a small displacement: C21 already records that. The new trace reaches an upstream machine-precision conversion even for a well-scaled root near the origin. Likewise, F06 is not another generic request to hash source files: the portable runner already does that, and a separate native definition-comparison tool already freezes inputs. The new issue is the logical mismatch between its specific temporal claim and its live-input execution model.

The full novelty ledger appears in Appendix A. Existing subjects such as cutoff conventions, native storage densification, insufficient refinement demands, semantic exponent grouping, branch-safe logarithm normalization, and missing native help pages are not repackaged as fresh findings. Their status is relevant context, but their detailed recommendations remain in the earlier registers. The novelty claim is relative to the inspected maintained inventory; it is not a proof that no similar sentence occurs in unindexed historical prose.

2.3 A compatibility result needs four coordinates

A useful compatibility assertion has at least four coordinates:

(operation and input class, mathematical contract, runtime/version, evidence scope).

For example, “a rational quadratic certificate was obtained” does not establish that a transcendental reference root has 50 trustworthy decimal digits. “Definitions match on Wolfram” does not establish that a missing Mathics builtin supplies the same algorithm. “The kernel exited successfully” does not establish that it evaluated a symbolic head. These distinctions are not paperwork: they directly determine whether a user may safely compose, refine, or numerically specialize an object.

3 Comparison with built-in Wolfram Language

3.1 Representation rather than a function-name contest

Native `SeriesData` stores exponents on a rational lattice, with indices and a denominator. Its surrounding expression can include logarithmic coefficients, other symbolic factors, and special functions. `InverseSeries` is not limited to ordinary linear-leading Taylor series: native examples include ramified inversion. It would therefore be misleading to attribute all noninteger powers or all logarithms exclusively to this repository.[35, 36]

The package instead makes a finite ordered collection of exact-real exponents and polynomial logarithmic coefficients an explicit working representation. In a positive local coordinate u , its ordinary finite part has the form

$$P(u) = \sum_{\alpha \in A} u^\alpha B_\alpha(\log u), \quad A \subset \mathbb{R} \text{ finite}, \quad B_\alpha \in \mathbb{R}[L], \quad (1)$$

with retained assumptions where coefficients depend on parameters. The error descriptor

$$R(u) = \mathcal{O}\left(u^\beta(1 + |\log u|)^k\right), \quad u \rightarrow 0^+, \quad (2)$$

is a separate piece of information, not merely typography on the last coefficient. Arithmetic must transport it together with the finite expression. The core explicitly distinguishes exact finite information from a finite working order and rejects a number of unsupported or unproved cases.[2]

An illustrative mathematical family is

$$f(x) = x + x^\alpha, \quad \alpha > 1, \quad x > 0.$$

Writing $x = yV(y^{\alpha-1})$ reduces the inverse equation to $V + tV^\alpha = 1$. The implicit function theorem at $(V, t) = (1, 0)$ gives an analytic $V(t)$ for each fixed real α , since the derivative with respect to V is one there. Substitution yields

$$f^{-1}(y) = y - y^\alpha + \alpha y^{2\alpha-1} - \frac{\alpha(3\alpha-1)}{2} y^{3\alpha-2} + \mathcal{O}(y^{4\alpha-3}).$$

When α is irrational, the exponents are not all on a rational lattice in y . A structured generalized jet is then a natural representation. This derivation is an independent mathematical example, not an assertion that a particular native command fails on it. Native software may obtain the expansion through substitutions or other representations.

3.2 Native asymptotics is broader than Taylor expansion

`Asymptotic` handles more than Taylor series: its documented scope includes non-power scales and asymptotic forms involving other computational objects. `AsymptoticSolve` treats equations and systems with a small or large parameter, so inverse asymptotics can often be posed natively as an asymptotic solving problem. Neither function should be reduced to a call to `InverseSeries`. The documentation also gives order, condition, and precision conventions that must be matched before comparing outputs.[37, 38]

The following matrix describes roles, not a benchmark or an exhaustive decision procedure for native solvability.

Task	Built-in route	Repository contribution / boundary
Regular local expansions	Series , structured SeriesData .	Retained positive-real chart, source, complete blocks, and explicit power-log remainder. Native work should remain the baseline for ordinary cases.
Local inversion	InverseSeries ; asymptotic solving of an implicit equation.	Multi-index generalized coefficients, selected real inverse germs, retained inverse models, and subsequent operations on the result.
Irrational power-log support	Symbolic expressions, transformations, and supported native asymptotics.	Explicit sparse exact-real exponent ordering with logarithmic polynomial blocks; ordering must be provable.
Logarithmic or exponential leading cores	Native special functions and general asymptotic engines.	Dedicated inverse-logarithmic brackets and exact-core marker expansions; different cutoff meanings are recorded by scale.
Oscillatory logarithmic coefficients	Native expressions and function-specific asymptotics.	A finite Fourier-polynomial coefficient algebra with conjugacy checks and a frequency budget. This is not arbitrary oscillatory asymptotics.
Gamma / Barnes families	Native special functions, asymptotics, and numerical evaluation where supported.	Dedicated forward and inverse normalizations, retained cores, and finite-model diagnostics. Their analytic and numerical scopes remain separate.
Error-aware later operations	Native structured-series arithmetic within its semantics.	Explicit transport of the stored remainder through package operations, with conservative refusal outside admitted contracts.
Numerical validation	Native arbitrary-precision solvers and numerical facilities.	Reference-root comparison and selected exact interval certificates. F01 shows why the same numerical request is not automatically portable to Mathics.
Native compatibility	The native function itself.	Explicit/automatic delegation preserves a native payload without automatically assigning an analytic package remainder. A native wrapper is not a new theorem.

The repository-side statements follow the public contracts and the inspected engines.[2, 4, 5, 6, 8] A fair comparison must request the same branch and achieved information. A native order, a target-exponent cutoff, a number of complete nonzero blocks, and a marker degree can all be spelled as an integer while meaning different things. Earlier reviews already analyze this API issue; it is not counted again here.[26, 27]

3.3 Four levels of evidence about an inverse

For an approximation $g(y)$ to $f^{-1}(y)$, four superficially similar outputs answer different questions.

A finite formal residual checks an algebraic coefficient identity in an admitted model. It does not, by

itself, bound the error of the original function outside that model. An asymptotic remainder states eventual behavior, usually with an unspecified constant and threshold. A high-precision reference comparison is numerical evidence and can fail through branch selection, conditioning, precision loss, or an upstream evaluator. A rational interval certificate supplies explicit enclosures and verified hypotheses for a particular numerical target.

The package’s API and source recognize these distinctions. For example, ordinary `InverseResidual` describes a finite forward-model composition, whereas `InverseNumericalCheck` solves a retained equation and `InverseCertificate` has a separate interval contract. This separation should be preserved while repairing the Mathics numerical route: an unsupported reference-root calculation should not invalidate an independently correct symbolic expansion, and a small residual should not be silently promoted to a certificate.[2, 3, 9]

3.4 Assessment of performance claims

No runtime comparison between the repository and Wolfram is justified by the present session. Existing binary powering, sparse product pruning, grouped Lagrange methods, Newton scheduling, and refinement caches must be recognized as implemented mechanisms, not proposed as new features. Their unresolved precision or resource questions are already tracked.[26]

For the present findings, performance evidence can be much more targeted: count all positions materialized by the Mathics `FirstPosition` helper, measure retained diagnostic bytes, distinguish cold process/load time from symbolic work, and record actual numerical precision rather than requested precision. A faster wrong-branch evaluation or a nominally successful machine-precision result at a 50-digit request is not a speedup. The artifacts supplied here measure only independent mechanisms, not package throughput.

4 Mathics compatibility: substantial work, incomplete equivalence

4.1 The implementation is a scoped semantic adapter

The early compatibility context is inserted while kernel definitions are parsed, so selected names bind to package-owned adapters. It is removed from the public context path after loading. Later adapters rewrite particular private definitions. The documented design leaves existing `System` definitions untouched on Mathics and leaves the official Wolfram route isolated from the compatibility context. The creation of inert `System` names for missing public symbols is deliberate name alignment, not an implementation of those functions.[2, 10, 9]

Several repairs solve genuine evaluator differences. The module/return adapter provides a per-invocation dynamic tag so an explicit module return crosses loops correctly. Association lookup preserves lazy defaults and lists of keys. Held extraction supports named callable parameters without prematurely evaluating them. Finite derivative sums use tables to bind indices before evaluation. An empty-list mapping adapter avoids a documented interpreter cache failure. These mechanisms are evidence of a serious port, rather than a superficial claim that both systems understand bracket syntax.[10, 11, 15, 19]

The assumption layer also has an appropriate limited aim. It derives selected finite-real and sign consequences from explicit facts, retains unknown cases, and protects unresolved ordered comparisons from native simplification that could erase a complex offset. It is not advertised as quantifier elimination. The inverse-branch layer admits selected polynomial and affine-domain proofs without claiming a complete replacement for `Reduce`. Those conservative refusals are preferable to guessed

branches.[12, 13, 17]

4.2 Capability should be stratified

Layer	Existing evidence / implementation	Remaining boundary relevant here
Load and namespace	Modular/standalone adapters, reload checks, context isolation.	Installation success does not establish support for every inert symbol; late rewrites need their own invariants.
Exact algebra	Association, return, finite-sum, trigonometric and exact-number adapters.	Helper grammar and evaluation cost still differ in F03/F04; algebraic normalization is more limited.
Assumptions and branches	Bounded exact sign reasoning and selected convex real-domain proofs.	Fixed neighborhood trials miss readily solvable rational affine cases; unsupported general proofs should remain unresolved.
Analytic expansions	Defining Taylor series and a finite positive-argument Stirling model supplement native functionality.	Parameter ranges and terminating exceptions must be distinguished; a finite Poincare tail is not exactness.
Numerical evidence	Selected principal Lambert and quadratic smoke examples.	F01/F02 show distinct precision and branch-dependency hazards beyond these controls.
Certificates	Selected rational interval examples and a history-index adapter.	This evidence does not certify all special-function inverses or all numerical targets.
Native delegation	A separate native-result contract is present.	Delegation can only use algorithms actually implemented by the runtime; an inert Asymptotic head is not coverage.

This matrix synthesizes the implementation and the project’s own scoped status statements.[9, 14, 18, 20] It is not a newly executed acceptance matrix.

4.3 What the repository’s receipts establish

The compatibility document reports a Wolfram preservation run with 1,452 passes and 12 pre-existing failures across 79 suites, with all 1,464 per-test records matching an untouched 6687962 baseline. It separately describes later native definition comparisons, including a comparison against an a55df16 control for the Mathics candidate. Those comparisons cover 2,051 modular and 2,050 standalone package symbols and selected builtin/behavior controls. The document correctly avoids relabeling the earlier full behavioral run as a full run of subsequent source changes.[9]

The current portable workflow is configured to run grouped fresh-kernel tests for modular and standalone entries on Python 3.11. Its runner rejects empty selections and malformed success protocols, records interpreter diagnostics, and uses process-level timeouts. Those are important improvements over earlier generic testing concerns. Nevertheless, workflow configuration is not evidence that a particular workflow execution completed successfully, and the documentation still marks consolidated final-source acceptance as pending.[24, 22, 23, 9]

A practical compatibility statement should therefore say: selected symbolic, exact-algebraic, and certificate examples have documented support; complete feature and parameter coverage is unestablished; the numerical paths in F01/F02 require focused work. Neither “Mathics is incompatible” nor “the package is Mathics-compatible” without qualification is an adequate engineering summary.

4.4 User-facing consequences

Streaming load order matters: a single command-line input containing both `Get` and a first call can be parsed before the package exports its symbols, binding previously unknown names in `Global`. The documentation already warns about this. Similarly, the test environment explicitly raises Mathics’ iteration limit without changing the package user’s global setting. These are documented setup constraints, not newly discovered bugs.[9, 23]

The actionable UX improvement specific to this review is more precise numerical failure reporting. A message such as “root not found” is insufficient when the actual obstacle is unsupported working precision; a complex value from a known incorrect nonprincipal dependency should not look like an ordinary evaluated real core. Runtime-specific capabilities should be attached to the numerical operation being attempted, not used to brand an otherwise valid symbolic object as wholly unsupported.

5 F01: the high-precision root route is not portable

Priority: high. Evidence: source-proved upstream precision reduction and package postcondition gap. End-to-end package behavior remains unexecuted. This finding is distinct from loss of a small displacement at a huge translated endpoint.

5.1 The data-flow mismatch

The ordinary numerical checker validates the requested precision, requires an exact target or sufficiently precise input, evaluates the finite approximation at extra precision, and calls `FindRoot` with a precision-oriented request. Its returned association describes a high-precision reference comparison. The core request has this structure:[3]

```
FindRoot[eq == 0, {xx, start},
  WorkingPrecision -> wp + 10,
  AccuracyGoal -> Infinity, PrecisionGoal -> wp,
  MaxIterations -> 500]
```

In Mathics 10.0.1, the inspected `_BaseFinder` implementation does not list `WorkingPrecision` among its options. More decisively, it initializes the seed with `eval_N(x0, evaluation)`. The default precision argument of that helper is `SymbolMachinePrecision`; its numerical path rounds a `Number` to the requested precision. The next two links remove any ambiguity: `get_precision` maps the machine-precision marker to `None`, and `PrecisionReal.round(None)` constructs `MachineReal(float(self.value))`. Newton helper paths also invoke `eval_N` without transporting the package’s requested working precision.[29, 31, 32, 33, 30]

The mismatch is thus structural. Constructing a 60-digit seed in the caller does not preserve 60-digit arithmetic when the callee rounds it to machine precision. Reapplying `N[root, 50]` afterward cannot recover lost information. Padding precision would alter a tag, not supply new correct digits.

There are two possible externally visible outcomes that must not be conflated. Unsupported-option messages or solver failure may make the package’s `Check` return a safe failure. Alternatively, some

route may return a numerical root that passes `NumericQ`, realness, and source-domain checks without meeting the intended precision. The source audit establishes the unsupported precision pipeline; it does not establish which of these outcomes a fresh interpreter produces for every example.

5.2 Why current smoke tests are weak controls for this issue

An exactly representable root, a special value simplifying before numerical work, or a test that only checks a value to ordinary display precision can pass despite this defect. A quadratic example with an exact root is useful for control flow but is not a stress test for arbitrary precision. The correct comparison should use a well-scaled irrational or transcendental root and inspect both the achieved numerical precision and an independently evaluated residual. The project’s compatibility document already limits its numerical smoke evidence; that qualification should be maintained.[9]

The unexecuted probe suite includes a direct `Exp[x]==2` root request with a high-precision seed and a public inverse check based on `Exp[x]-1`. These avoid the huge-source-origin issue in C21. The latter uses an exact target $1/10$, whose true source root is $\log(11/10)$, so loss of target input digits is not the explanation.

5.3 Repair and acceptance criteria

The first repair should be a numerical capability boundary, not a cosmetic option substitution. At the call site, distinguish unsupported requested precision from nonconvergence and from an invalid branch. On return, inspect actual precision/accuracy before publishing a high-precision comparison. A check of a metadata precision alone is insufficient: residual evaluation must avoid treating an inexact zero with poor accuracy as exact agreement.

For `Mathics`, a precision-preserving solver requires an implementation that explicitly carries arithmetic precision through seed creation, derivatives, residuals, updates, and stopping decisions. Alternatively, route supported equations through a package-owned rational interval method. The ordinary Wolfram route should remain unchanged. Rejecting an unsupported numerical request does not require discarding the exact series object.

Focused acceptance should include requests at 20, 50, and 100 digits, both finite and infinite source charts, a genuine transcendental root, a failed-convergence control, and exact-root controls. Report requested precision, achieved precision, residual accuracy, and the precise numerical evidence scope. A native runtime should act as a comparison, not as the sole oracle for the exact branch or mathematical target.

6 F02: retained nonprincipal Lambert cores remain numerically hazardous

Priority: high for affected numerical specialization. **Evidence:** source-traced dependency escape and an executed `mpmath` countermodel; `Mathics` package output `unrun`. The upstream defect is already acknowledged by the repository. The new contribution is the concrete retained-core route and a narrow exact fallback.

6.1 What is fixed and what still escapes

The `Mathics` core-function adapter correctly recognizes a dangerous argument-order mismatch in the runtime’s two-argument `ProductLog`. It rewrites a principal-branch construction to the one-argument spelling, but a nonzero branch is returned as active `System‘ProductLog[branch,argument]`. The

upstream `ProductLog` class delegates to `mpmath`'s `lambertw`, whose argument order is `lambertw(z,k)`, rather than Wolfram's `ProductLog[k,z]`. The adapter explicitly declines to provide nonprincipal numerical evaluation.[\[16, 34, 42\]](#)

That documented limitation becomes more serious when an exact core is retained in the finite expression. `corePerturbationAutomaticInverse` constructs the branch expression, and with zero perturbation the core itself is the returned finite approximation. The public numeric application of a nonnative `GeneralizedSeries` performs substitution into its `Expression`; it does not consult the interpreter's special-function capabilities. An outer `N` can therefore evaluate the active dependency.[\[5, 2\]](#)

A targeted public candidate is:

```
s = AsymptoticCoreInverse[-x Log[x], 0, {x, 0}, {y, 0}];
N[s[1/100], 50]
```

This is a proposed fresh-kernel characterization, not a transcript from this session.

6.2 The mathematical branch is unambiguous

On $0 < x < e^{-1}$, $-x \log x$ is strictly increasing from zero to e^{-1} . Its small positive inverse is

$$x(y) = \exp(W_{-1}(-y)) = -\frac{y}{W_{-1}(-y)}, \quad 0 < y < e^{-1}. \quad (3)$$

Indeed, putting $w = \log x < -1$ gives $we^w = -y$. The smaller positive x requires the branch $w < -1$, not the principal branch. The branch convention agrees with the primary `mpmath` documentation.[\[42\]](#)

At $y = 1/100$, the independent high-precision `mpmath` calculation recorded in the archive gives approximately

$$\begin{aligned} W_{-1}(-0.01) &= -6.472775124394004694741057892724488\dots, \\ x(0.01) &= 0.00154493239882734560172624674508665\dots \end{aligned}$$

Passing the arguments in the wrong order instead produces, in that independent `mpmath` experiment,

$$\text{lambertw}(-1, -0.01) \approx -0.3181315052047641 + 1.3372357014306894i.$$

The resulting quotient is approximately

$$0.001683763790872229 + 0.007077541887847276i,$$

which cannot be the real small inverse. These values demonstrate the consequence of the dependency's argument permutation; they are *not* alleged `Mathics` output. A runtime may also reject or transform the call before reaching this numerical evaluation.

6.3 Do not overgeneralize the witness

The logarithmic Lambert engine often returns a finite inverse-logarithmic bracket expressed in elementary logarithms, keeping an exact Lambert expression only in metadata. Such a finite bracket need not call `ProductLog` during numerical evaluation. The retained exact-core route is different: its finite expression can contain the active core function. Therefore this finding is not a claim that every Lambert expansion or every nonprincipal symbolic result is invalid.[\[4, 5\]](#)

It is also not a rediscovery that `s[value]` is generally an unchecked finite substitution; earlier reviews already discuss that policy. The sharper point is that a specifically known incorrect numerical backend can be reached by an advertised exact-core output even when the target lies on a perfectly valid real branch.

6.4 Repair before evaluation, not after it

A guard must act while the branch and argument remain symbolic or held. Detecting a `ProductLog` head after substitution can be too late: evaluation may already have replaced the head with a wrong number. A package-owned exact-core descriptor or carefully held dispatch can preserve branch information until a supported numerical adapter is selected. Unsupported branches should yield a specific capability failure, not an apparently ordinary complex value for a real branch.

For the bounded family in (3), Section 10 supplies an independent rational certificate avoiding `ProductLog` altogether. This does not solve general Lambert evaluation, complex branches, or the coalescing-branch region near e^{-1} . It is a concrete alternative for a useful real subdomain, with a much smaller numerical trust boundary.

7 F03: default limits change mathematical meaning

Priority: medium; helper conformance defect. A wrong public branch has not been reproduced. An inspected branch-validation consumer explicitly supplies the correct positive-local-coordinate direction and is not affected by the omitted-direction case.

The Mathics `Limit` wrapper declares `Direction -> Automatic` and then replaces `Automatic` with `+1`. Its explicit string conversions are consistent with the supported integer convention: `"FromAbove"` maps to `-1` and `"FromBelow"` to `+1`. However, current Wolfram `Limit` documents a two-sided real default, `Direction -> Reals`. Mapping an omitted direction to `+1` changes the question rather than merely translating syntax.[13, 39]

For $f(x) = |x|/x$ at zero, the two one-sided limits are `-1` and `+1`, so the two-sided limit does not exist. The helper's default instead requests the left-sided value. The independent mathematical distinction is immediate. The supplied probe compares default, `-1`, and `+1` forms on the two runtimes.

The source inspection also provides a useful negative control: `inverseBranchValidate` first maps the source to a positive local coordinate and explicitly calls `Limit[... , u->0, Direction->-1, ...]`. Its branch-validation limit therefore does not rely on the defective helper default.[7] This prevents a helper-level discrepancy from being overstated as an established wrong inverse result.

The smallest repair is to narrow and enforce the adapter's admitted grammar: require an explicit supported one-sided direction at internal call sites, and refuse unsupported forms. A broader repair can implement the documented two-sided request by obtaining both justified one-sided values and combining them only when equality is established. Unknown or incompatible outcomes must remain unresolved or indicate nonexistence as appropriate; they must not default to one side. Infinite endpoints and complex directions need their own admission rules rather than being swept into the finite two-sided case.

This is distinct from the earlier public direction-alias work. The problem is the default meaning of an internal operation. Before broadening the adapter, inspect all consumers and retain explicit-positive-coordinate tests so a semantically correct default does not accidentally alter code that already chooses its side deliberately.

8 F04: first-position grammar and avoidable traversal

Priority: medium; helper correctness and structural resource cost. The option-only calls and predicate-count characterizations have not been run in Mathics here. No current large public workload is claimed to reproduce the resource issue.

8.1 An option is consumed as a default value

The compatibility definitions provide two-, three-, and extended four-positional-argument forms. The three-argument form treats its final argument unconditionally as the missing-match default. Only the extended form uses `OptionsPattern`. Consequently, an ordinary option-only third argument can be misclassified. Native `FirstPosition` supports a `Heads` option as well as a separate default argument.^[10, 40]

Consider the intended calls:

```
FirstPosition[{a}, b, Heads -> False]
FirstPosition[h[a], h, Heads -> False]
```

In the inspected helper, the first call supplies the expression `Heads -> False` as the held default while still searching heads. If no match is found, that rule is returned as data instead of the ordinary missing result. The second likewise searches heads despite the apparent option. A consumer can receive a syntactically plausible value of the wrong kind, making this more confusing than a clear unsupported-argument refusal.

The repair needs explicit positional/option grammar, with overlapping definitions ordered or conditioned so an option rule is not accepted as a default by accident. Preserve the useful existing behavior: an effectful default must not run when a match exists. The optional levels argument and `Heads` setting should be tested together. This is a local primitive adapter defect, not a new report of the already cataloged general public option-resolution issues.

8.2 The first result is obtained by storing every result

The implementation calls `Position` without a match-count bound, stores the complete positions list, and then selects its first element. If there are M matches, it creates at least M position objects even though its public result contains at most one. For a list of n integers matched by `_Integer`, the result-storage component is linear in n where a first-match implementation can stop at the first admissible position. Nested paths add path-length storage. This is a structural cost deduction, not a measured speed ratio.^[10]

For a pure predicate, a bounded `Position` form is a natural candidate: native `Position` has a count-limited signature. Its actual support in Mathics must first be checked. If that form is unavailable, use a bounded traversal that preserves search order, level restrictions, head traversal, and lazy default evaluation.^[41] A replacement must not change the result in order to save work.

The supplied predicate-count probe also makes additional matching work observable. It is a characterization of evaluation behavior, not a claim that the native documentation guarantees an exact side-effect count for every pattern. The important optimization target is bounded search and storage under a clearly specified primitive contract.

9 Portable validation: three focused runner findings

The runner already does several things correctly: fresh processes, explicit test selection, protocol validation, exit-status matching, process-tree termination, and a frozen copy of the suite text. These findings refine that implementation rather than repeating the old observation that a legacy aggregate runner could accept an empty suite.^[22, 23, 26]

9.1 F05: nonfinite timeouts pass the input guard

The command-line timeout is parsed as a Python float and rejected only when it is less than or equal to zero. Both

```
float("nan") <= 0
float("inf") <= 0
```

evaluate to **False**. The independent tests execute that exact predicate. The intended contract—a finite positive process deadline—therefore is not enforced.^[22]

The later effect of passing NaN or infinity into subprocess waiting can vary with Python/platform implementation: it may raise, produce an unusable wait, or fail while converting a deadline. No particular original-runner exception was observed in this session. The defect is established before that point: accepted input does not satisfy the documented resource contract.

Validate with `math.isfinite(value)` and `value > 0` during argument parsing, before launching a child. Include a diagnostic naming the invalid value and an independent test for NaN, positive/negative infinity, zero, negative values, malformed strings, and a valid fractional timeout. `code/bounded_process.py` supplies the tested validator. An output-byte budget also needs a positive integer check.

9.2 F06: boundary equality is not temporal source immutability

The portable runner fingerprints the selected source and related files before execution, compares them with later fingerprints, and records **SourcesUnchangedDuringRun**. It copies the suite bytes into a temporary file, but passes the live package path to each fresh kernel.^[22]

Let A denote original file bytes and B changed bytes. The sequence

$$A \longrightarrow B \text{ (consumed by a kernel)} \longrightarrow A$$

passes a comparison of the first and last observations. Even cryptographically strong digests do not distinguish that sequence from a file that stayed at A throughout. The issue is not a collision; it is an unobserved temporal interval. Repeated post-case checks narrow but do not eliminate the gap.

The archive includes an executed byte-level countermodel of this sequence. It is not evidence that any repository receipt was corrupted or that a contributor edited files during a run. It establishes that the runner's field name asserts more than its measurements prove.

A stronger implementation should execute a snapshot whose bytes are chosen independently of the mutable working tree. The supplied `code/run_audit_probes.py` reads regular Wolfram source blobs from a full Git commit ID into a temporary package tree. The executed synthetic Git test changes the live entry and deletes its helper after committing, then confirms that both modular and standalone snapshots contain the original committed bytes. No real repository kernel is needed to validate that mechanism.

This approach should be integrated with the project's existing snapshot practice rather than introduced as if it were novel repository-wide infrastructure: the separate native definition-comparison tool is already documented as copying package inputs and its capture script.^[9] For the portable runner, freeze the package, the suite, and the test-discovery input coherently. Record the selected commit and actual execution paths. Merely changing a permissions bit is not a security boundary against a malicious same-user process; the aim here is reproducibility against ordinary source edits, not sandbox security.

If live execution remains available for development, name the observation accurately, for example

“pre/post source digests equal,” and do not use that mode to claim immutable-input acceptance. Reject a modified frozen script or changed snapshot before subsequent kernel launches when such integrity checks are part of the chosen protocol.

9.3 F07: diagnostics have no memory budget

Each kernel’s combined output is collected through `subprocess.communicate`, retained in full as `KernelOutput`, and included in the accumulated JSON report. There is no independent byte cap on that channel. A symbolic term budget does not bound diagnostics, and a finite wall-clock deadline does not bound memory without a bound on output rate.[22]

There is a second structural cost. If n cases each contribute approximately B bytes and the complete accumulated report is serialized after every case, total serialized transcript volume is proportional to

$$B(1 + 2 + \cdots + n) = \Theta(n^2 B).$$

This is a work model for the repeated-write design, not a claim about measured runtime on the current suite. Small bounded logs may make the cost negligible; unbounded logs make it an avoidable failure amplifier.

The supplied POSIX supervisor retains at most a configured number of bytes, distinguishes output-limit exhaustion from a timeout, and terminates only its own process group. Tests include a child producing two million bytes with a 16,384-byte retention cap, a sleeping child, UTF-8 output, nonzero exit preservation, and launch failure. The implementation does not invoke a shell. It is intentionally not a Windows replacement; a Windows implementation must integrate a reliable owned-process-tree mechanism and test launcher/child pipe behavior separately.

For repository integration, preserve the protocol and distinguish `OutputLimit`, `Timeout`, `KernelError`, and a mathematical assertion failure. Truncated output must never count as a complete success record. A capped diagnostic file plus a compact per-case summary avoids repeatedly embedding all historical transcripts. Keeping a prefix and suffix can aid diagnosis, but the discarded-byte count and truncation status must remain explicit.

10 A proved, executable fallback for a small Lambert inverse

The general proposals for quantitative special-function bounds and independent checkers already appear in earlier reviews. The contribution here is narrower and executable: exact rational enclosures for the small inverse of $-x \log x$ on a declared target range. It directly addresses the numerical dependency in F02 without relying on either Mathics or mpmath for proof.[26]

10.1 A rational enclosure for the logarithm

Lemma 10.1. *Let $1 \leq v \leq 2$, $z = (v - 1)/(v + 1)$, and $N \geq 1$. Then*

$$S_N(v) = 2 \sum_{j=0}^{N-1} \frac{z^{2j+1}}{2j+1} \leq \log v \leq S_N(v) + \frac{2z^{2N+1}}{(2N+1)(1-z^2)}.$$

All endpoints are rational when v is rational.

Proof. For $0 \leq z \leq 1/3$, integrate the geometric series for $2/(1 - z^2)$ from zero to z to obtain

$$\log \frac{1+z}{1-z} = 2 \sum_{j=0}^{\infty} \frac{z^{2j+1}}{2j+1}.$$

The omitted summands are nonnegative. In the tail, replace every denominator by the smaller $2N + 1$ and sum the resulting geometric progression. This proves the upper bound; the identity $v = (1 + z)/(1 - z)$ completes the result. $\square$

For any positive rational q , choose an integer k such that $q = 2^k v$ with $1 \leq v < 2$. The identity

$$\log q = k \log 2 + \log v$$

reduces enclosure to two applications of Lemma 10.1. When $k < 0$, multiplying the interval for $\log 2$ reverses its endpoints. The implementation performs that sign-sensitive interval operation explicitly. The tail decreases geometrically after range reduction, independently of the magnitude of q , apart from the factor $|k|$ multiplying the uncertainty in $\log 2$.

10.2 A uniformly valid initial bracket

Proposition 10.2. *For every rational $0 < y \leq 1/4$, the equation*

$$-x \log x = y$$

has exactly one solution in $(y^2, 1/4)$. It is the small real inverse in (3).

Proof. Let $F_y(x) = -x \log x - y$. On $0 < x \leq 1/4$,

$$F'_y(x) = -\log x - 1 \geq \log 4 - 1 > 0.$$

Thus F_y is strictly increasing on the interval. At the upper endpoint,

$$F_y(1/4) = \frac{\log 4}{4} - y \geq \frac{\log 4 - 1}{4} > 0.$$

For the lower endpoint,

$$F_y(y^2) = y(2y \log(1/y) - 1).$$

The function $y \log(1/y)$ is increasing on $(0, 1/4]$, because its derivative is $\log(1/y) - 1 > 0$. Therefore

$$2y \log(1/y) \leq \frac{1}{2} \log 4 < 1,$$

so $F_y(y^2) < 0$. The inequalities $1 < \log 4 < 2$ can themselves be verified by the rational logarithm bounds; they are checked by the implementation. Continuity and strict monotonicity give existence and uniqueness. Since $1/4 < e^{-1}$, this is the small branch. $\square$

10.3 Bisection with exact sign decisions

At a rational midpoint m , an enclosure $\log m \in [L, U]$ gives

$$F_y(m) \in [-mU - y, -mL - y].$$

If the upper endpoint is negative, the root is to the right; if the lower endpoint is positive, it is to the left. Otherwise the sign is unresolved at the current logarithm order and the order is increased. No numerical sample or unproved sign is used to move a bracket endpoint.

The implementation has explicit bisection and logarithm-order budgets. Exhaustion yields an error rather than a claimed certificate. Hence the following result is a soundness statement for returned certificates, not a proof that arbitrary resource settings always suffice.

Theorem 10.3. *Suppose the supplied verifier accepts a certificate for rational $0 < y \leq 1/4$ with interval $[a, b]$ and requested width 2^{-p} . Then the small real inverse $x(y)$ lies strictly between a and b , the interval width satisfies $b - a \leq 2^{-p}$, and*

$$W_{-1}(-y) \in \left[-\frac{y}{a}, -\frac{y}{b} \right].$$

Proof. The verifier checks the declared domain and interval ordering, a positive derivative lower bound, and strict opposite endpoint signs using rational logarithm enclosures. By monotonicity and the intermediate value theorem, exactly one root lies between the endpoints. It also checks the width inequality directly in rational arithmetic. Finally, $-y/x$ is strictly increasing in positive x , and (3) identifies its value at the root with $W_{-1}(-y)$. Applying this increasing map to the bracket proves the last enclosure. $\square$

10.4 Executed result and its precise scope

For $y = 1/100$ and an 80-bit *source-width* request, the supplied run returned

$$\begin{aligned} a &= \frac{4669271666253859866075463}{3022314549036572936765440000}, \\ b &= \frac{2334635833126929933038981}{1511157274518286468382720000}, \\ b - a &= \frac{2499}{3022314549036572936765440000} < 2^{-80}. \end{aligned}$$

It used 78 bisections and a final logarithm order of 24. The stored verifier result is true. These are executed observations from the independent implementation, retained in `evidence/entropy-certificate.json`.

The width goal is for x , not for $W_{-1}(-y)$. Indeed,

$$\frac{y}{a} - \frac{y}{b} = \frac{y(b-a)}{ab},$$

so a Lambert-value error goal needs its own stopping rule or an appropriately stronger source goal. The artifact records both widths to prevent a precision-labeling mistake similar in spirit to F01.

This checker trusts Python’s exact integer/rational arithmetic and its own code; it is not a proof-assistant formalization. Its verifier operates on the typed certificate object, not on arbitrary hostile JSON. It does not certify the other real branch, targets above $1/4$, complex branches, all special functions, or the package’s own interval implementation. The tests include mutated interval, target, and width claims that are rejected, but finite testing is not the mathematical reason for soundness: the preceding argument is.

11 Three concrete compatibility improvements

11.1 E01: solve the rational affine neighborhood problem instead of sampling radii

The Mathics eventual-condition helper simplifies under $u > 0$, tries the seven radii 2^{-j} for $j = 0, \dots, 6$, and then falls back to the original proof route. A failed trial is correctly not taken as proof of impossibility. But the fixed list adds unnecessary incompleteness in a fragment that can be solved exactly.[17]

For example, $0 < u < 1/1000$ holds in a sufficiently small deleted positive neighborhood, but no interval $(0, 2^{-j})$ in that list is small enough. The original fallback may still resolve the condition, so this is not an established public failure. It is a concrete opportunity to remove an avoidable search horizon.

Consider an affine constraint

$$a + bu > 0, \quad a, b \in \mathbb{Q}.$$

If $a > 0$ and $b < 0$, choosing $r \leq a/(-2b)$ proves positivity throughout $0 < u < r$. If $a > 0$ and $b \geq 0$, any sufficiently small positive radius works. If $a = 0$, strict positivity is equivalent to $b > 0$. If $a < 0$, the constraint is false throughout a sufficiently small positive neighborhood. Non-strict inequalities are handled with the corresponding zero cases, and negative inequalities by sign reversal.

Equality holds eventually precisely when both coefficients vanish. Unequality holds eventually when at least one coefficient is nonzero; when a nonzero intercept and slope can create a positive root, a radius below that root avoids it. For a finite conjunction, take the minimum of the radii supplied by all satisfiable constraints. This gives an exact decision procedure for this rational affine eventual-truth fragment, not for arbitrary logical domains.

`code/exact_portability_prototypes.py` implements these cases and returns $1/2000$ for the displayed example. Tests cover zero endpoints, equality, unequality, and eventually false conditions. A Wolfram implementation can use the same finite case distinction. Extension to algebraic or parameter-dependent coefficients should require proved sign/order comparisons and should retain an inconclusive result when those proofs are unavailable.

11.2 E02: admit terminating hypergeometric series before convergence screening

The defining Taylor adapter covers several hypergeometric functions and `PolyLog` at a vanishing monomial argument. Its guarded shape—one varying function times a constant, explicit order information, positive lower parameters, real upper parameters—avoids a number of unsafe surrounding-pole and domain cases. For a general `HypergeometricPFQ`, however, it applies the $p \leq q+1$ convergence restriction before considering termination.^[14]

A nonpositive integer upper parameter $-m$ makes the series terminate. Under the adapter’s existing positive-lower-parameter restriction, there is no denominator pole, and the result is a polynomial regardless of $p - q$. This exception can be admitted without introducing asymptotic summation of divergent nonterminating hypergeometric series.

For rational admitted parameters, let m be the smallest termination degree and set $c_0 = 1$. The finite recurrence

$$c_{n+1} = c_n \frac{\prod_i (a_i + n)}{(n+1) \prod_j (b_j + n)}, \quad 0 \leq n < m,$$

constructs every coefficient exactly. The supplied prototype returns

$${}_3F_1(-2, 3, 5; 7; z) = 1 - \frac{30}{7}z + \frac{45}{7}z^2.$$

The identity follows directly from the finite defining coefficients. It is independently tested in the archive.

The implementation should check the degree and term budget before allocation, retain the same parameter restrictions, and mark exact termination only when it has been proved. If a requested cutoff removes part of the polynomial, the discarded finite terms still contribute an error at that cutoff. The current native fallback may already evaluate some terminating inputs, which is why the supplied

public `terminating-pfq` probe is a characterization rather than a predicted universal failure. The improvement is a direct, portable coefficient path with known cost and exact-tail information.

11.3 E03: make late adapter rewrites assert their installation invariants

Several late adapters replace selected patterns in a consumer’s `DownValues`. For example, the refinement adapter targets an association built from a mapped rule list and another built from a part extraction. The later list adapter rewrites private references to `System`Map`. The current module order accounts for these dependencies.[21, 19, 2]

A future source refactor can make such a pattern match zero times without producing a load error. A changed spelling, an earlier rewrite, or an added helper can leave an apparently loaded package using the unsupported evaluator operation. This is a maintenance risk, not an allegation that the present adapter failed to install.

Each nontrivial rewrite should declare its consumer, expected pattern count, and a post-installation invariant. During tests, assert that the original unsupported form is absent from the intended site and that the replacement has the expected signature. Test reload idempotence and verify that the official Wolfram definition capture remains unchanged. Prefer explicit helper calls in new source when they are clearer than growing a collection of structural rewrites, but do not perform a broad refactor without preserving the currently documented isolation.

This recommendation is distinct from wave-3 W3-11’s standalone dependency scanner. One checks whether an artifact has external loading dependencies; the other checks whether a runtime-specific semantic patch actually attached to its intended consumer.

12 Development priorities and acceptance plan

The earlier registers already contain a wide mathematical roadmap. The following sequence is limited to the new mechanisms in this article and the minimal integration work they require.

12.1 Gate 1: numerical honesty before numerical breadth

First establish the realized-precision behavior of `InverseNumericalCheck` on the pinned `Mathics` runtime. An unsupported working-precision request must produce a specific outcome, not a mislabeled reference result. Independently characterize retained branch-1 cores and prevent active unsafe specialization. These changes have higher priority than increasing the special-function name list because they protect the interpretation of already returned objects.

Acceptance requires actual fresh-kernel transcripts for the direct root probe, public numerical probe, and retained-core probe, together with exact-domain and wrong-branch controls. The independent entropy certificate may serve as an oracle in its admitted target range, not as a substitute for testing the package’s dispatch and messages.

12.2 Gate 2: helper contracts and bounded primitive work

Repair the `Limit` default policy and `FirstPosition` option grammar. Preserve explicit one-sided branch consumers, lazy defaults, nested-level searches, and head exclusion. Establish whether `Mathics` supports a count-bounded `Position` before using it. Run the same semantic fixtures against official Wolfram to compare results, while allowing a deliberately narrower adapter to refuse unsupported forms clearly.

The performance acceptance for first-position work should count visited/matched nodes and retained positions at growing input size. Do not infer a speedup from a rewritten line of code. Measure ordinary and nested expressions separately and keep the requested search semantics unchanged.

12.3 Gate 3: acceptance records must describe the executed bytes

Admit only finite positive deadlines; bound diagnostic output; run the portable suite and package from coherent frozen inputs. Validate the supervisor on the supported operating systems before integrating it. Keep mathematical failure, interpreter exception, output exhaustion, timeout, protocol failure, and changed-input invalidation separate in the report.

The supplied POSIX runner demonstrates one implementation route. Its synthetic Git test and process tests establish those mechanisms only. Windows support, official-kernel command-line variations, and actual Mathics transcripts remain acceptance obligations. Freezing a Git commit is particularly useful for this repository because standalone and modular sources can then be compared at the same exact revision without racing development edits.

12.4 Gate 4: bounded extensions with independent oracles

Implement rational affine radius synthesis and terminating-PFQ recognition as small guarded additions. Their source-level proof obligations are short enough to state completely. Reuse the supplied exact prototypes as coefficient/radius oracles, but also test public caller behavior, preserved assumptions, failure cases, and resource limits. Do not broaden the inputs to symbolic coefficients or singular lower parameters merely because the rational prototype works.

An adapter-installation manifest should accompany such changes. That will make future refactors easier to audit and reduce dependence on undocumented parse/rewrite ordering. It is a specific addition to the existing native-preservation discipline, not a new argument for a repository-wide testing infrastructure that already exists.

13 Artifact guide and reproducibility

13.1 What the archive contains

Path	Purpose / execution status
<code>article.tex</code> , <code>article.pdf</code>	This article and its editable source.
<code>code/certified_entropy_inverse.py</code>	Exact rational logarithm bounds and a verified small inverse of $-x \log x$; executed.
<code>code/exact_portability_prototypes.py</code>	Rational affine eventual-radius solver and terminating PFQ coefficients; executed.
<code>code/bounded_process.py</code>	Bounded POSIX child-process supervisor and finite-timeout validator; synthetic tests executed.
<code>code/run_audit_probes.py</code>	Frozen-Git-input runtime probe driver; snapshot mechanism tested, real kernel execution not performed.
<code>code/PortabilityProbes.wl</code>	Eight fresh-kernel characterizations; unexecuted. Completion is not an assertion of mathematical success.

Path	Purpose / execution status
code/audit_checks.py	The 25-test independent suite. Requires mpmath for numerical cross-checks and Git for the snapshot fixture.
evidence/independent-checks.*	Actual transcript and summary of independent execution.
evidence/entropy-certificate.json	Actual 80-bit source-width certificate for $y = 1/100$.
evidence/lambert-argument-countermodel.json	mpmath argument-order countermodel, explicitly not Mathics output.
evidence/novelty-ledger.csv	Finding-to-prior-work crosswalk and evidence classification.
README.md, build.sh	Running instructions, artifact scope, and PDF build command.

13.2 Run the independent tests

From the archive root, in a suitable Python environment:

```
python -m pip install mpmath==1.3.0
cd code
python audit_checks.py
python certified_entropy_inverse.py 1/100 --bits 80 \
  --output ../evidence/entropy-certificate.json
```

The first command is an installation instruction for the user's environment, not a successful installation performed in this session. The independent test run recorded here used the already installed mpmath 1.3.0. The production exact certificate itself uses only the Python standard library.

13.3 Run package characterizations on a machine with a runtime

The Git clone must already contain the pinned commit; the driver does not fetch the network. On POSIX, use an installed Mathics environment:

```
python code/run_audit_probes.py /path/to/Asymptotic \
  --mathics-python /path/to/mathics-env/bin/python \
  --entry modular --timeout 180 \
  --output mathics-modular-probes.json
```

Repeat with `-entry standalone`. For an official kernel executable, replace the runtime selector:

```
python code/run_audit_probes.py /path/to/Asymptotic \
  --wolfram /path/to/WolframKernel \
  --entry modular --output wolfram-probes.json
```

The selector expects a kernel executable, not `wolframscript`; adapt the command explicitly for a different launcher. Run individual probes with `-case`. The driver records completion and output, not pass/fail comparisons against a universal expected string. Some probes intentionally call private Mathics helpers to isolate adapter semantics.

The public negative-Lambert and root-precision probes are the highest-value next experiments. The

affine and terminating-PFQ probes may demonstrate that a fallback already succeeds; that would narrow the opportunity rather than invalidate the source observation. The article’s findings should be updated with actual outcomes instead of treating every unexpected value as confirmation.

13.4 What was not delivered

No wholesale production patch is represented as validated. The Wolfram-language probes have not been syntax/evaluation-tested by a real kernel here. The independent POSIX supervisor has not been ported to Windows. No end-to-end performance ranking, current full-suite pass rate, or complete Mathics compatibility claim is supplied. These limits are intentional evidence boundaries, not a claim that the remaining work is impossible.

14 Conclusion

The repository’s strongest contribution is a retained asymptotic computation object with specialized real scales and error transport. Native Wolfram functions remain important both as complementary engines and as controls; reducing the comparison to Taylor inversion would undervalue the native system and obscure the package’s actual strengths.

The most consequential new result of this audit is that exact symbolic correctness and portable numerical correctness currently have different boundaries. Mathics’ root-finder precision pipeline and its nonprincipal Lambert dependency require explicit handling at the point of numerical use. Internal `Limit` and `FirstPosition` adapters reveal smaller but concrete semantic gaps. The runner findings show that reproducibility and bounded execution also need contracts as precise as the mathematical objects they validate.

The accompanying exact certificate, bounded process supervisor, frozen-input driver, and two small algebraic prototypes make those recommendations testable. Their successful independent tests do not close package acceptance; they provide controlled building blocks and oracles for it. The next useful milestone is not a broad compatibility label, but a set of fresh-kernel, pinned-source records establishing the repaired numerical and helper contracts on both runtimes.

A Novelty and overlap ledger

ID	Related prior item	Delta retained in this article
F01	C21; D05; R6 A10	Upstream Mathics seed/iteration conversion to machine precision, independent of large translations; package realized-precision postcondition and specific fresh-kernel probes.
F02	D05; X05/X06; existing Mathics limitation	Trace from a recognized exact core to active nonprincipal numerical evaluation; explicit branch countermodel and an executable rational fallback on $0 < y \leq 1/4$. The known upstream bug itself is not claimed as newly discovered.
F03	D06	Internal omitted-direction semantics differ from current native two-sided <code>Limit</code> ; the inspected explicit-direction branch consumer is a control, not a claimed failure.
F04	P08/P06; W3-02	Specific <code>FirstPosition</code> adapter grammar returns an option as a default, and all-match construction defeats first-result storage bounds. Not another general public-options report.

ID	Related prior item	Delta retained in this article
F05	V02; P06	Nonfinite timeout acceptance in the newer portable Python runner. Empty-suite checks are already present and are not reported missing.
F06	V01	Temporal provenance overclaim in live-package execution despite frozen suite text; edit/revert countermodel and tested Git-blob snapshot mechanism. Native definition tooling already uses snapshots.
F07	P06; V01	Unbounded child diagnostics and cumulative transcript serialization, outside mathematical term limits; independently tested capped process supervision.
E01	X07; bounded Mathics proofs	Exact rational affine neighborhood synthesis replacing a seven-radius heuristic in a completely specified fragment. No claim that general fallback cannot succeed.
E02	X12; termination work	Guarded terminating-PFQ exception to a specific $p \leq q + 1$ Taylor admission check; finite rational recurrence, exact polynomial oracle, and explicit remaining public characterization.
E03	W3-11; native-preservation discipline	Consumer-specific installation invariants for late DownValues rewrites, distinct from standalone external-dependency scanning.

The related-item IDs refer to the maintained registers, not to unresolved findings being newly asserted by this article. The exact certificate is a concrete implementation under X05/X06, not a claim that the general idea of interval certificates or independent checking is novel.[26, 27, 28]

B Source locations and review coverage

Repository-relative source locations below refer to 7d1bc832895cc90a9b2a978b7b7684acab908bd2. Named functions are the primary anchors; line ranges are included for the particularly important sliced reads.

Source	Relevant inspected region
<code>src/Kernel/AsymptoticAnalysis.wl</code>	Lines 1–200: contracts/bootstrap; 530–700: native ingress and local chart; 1120–1280: accessors, residual and numeric dispatch; 1300–end: coefficient dispatch and module order.
<code>src/Kernel/NumericalInverseChecks.wl</code>	<code>numericalInverseEvidence</code> , source-domain check, numerical output fields.
<code>src/Kernel/CorePerturbation.wl</code>	First 200 lines, especially automatic core inversion and zero-perturbation expression construction.
<code>src/Kernel/LambertInverse.wl</code>	Logarithmic/exponential core extraction, <code>lambertConstruct</code> , finite bracket and exact-core metadata.
<code>src/Kernel/InverseFunctionBranches.wl</code>	First 170 lines, especially explicit-direction limit validation and monotonicity/domain checks.

Source	Relevant inspected region
<code>src/Kernel/FourierCoefficients.wl</code>	Coefficient representation, exact frequency/conjugacy checks and bounded convolution in the initial source region.
<code>src/Kernel/Mathics*.wl</code>	Compatibility, assumptions, calls, algebra, calculus, simplification, Taylor, inverse branches, core functions, lists, refinement and special-function adapters discussed in text. Formatting/time-budget behavior is also informed by maintained documentation, not a complete independent evaluation.
<code>validation/run_mathics_tests.py</code>	Full runner, including timeout validation, output protocol, process handling, suite snapshot and pre/post source comparison.
<code>validation/MathicsTests.wl</code>	Initial 145 lines: setup, primitive mapping, load/reload, association, position and selected assumption tests. No exhaustive new coverage count is inferred.
<code>docs/Mathics/COMPATIBILITY.md</code>	Full status/limitations and preservation-receipt discussion.
<code>docs/development/CODE_REVIEW_STATUS.md</code>	Maintained issue inventory, complete wave-1/2 crosswalk, existing fixes and future-work scope.
<code>docs/development/WAVE_3_INTAKE.md</code>	Complete wave-3 crosswalk and consolidated work/proposal descriptions.

The upstream precision trace is anchored at Mathics3/mathics-core tag 10.0.1: `mathics/builtins/numbers/calculus.py`, lines 540–740; `mathics/eval/numbers/calculus/optimizers.py`, lines 250–430; and `mathics/eval/nevaluator.py`, lines 1–140. The precision trace continues through `mathics/core/number.py`, lines 40–135, and `mathics/core/atoms/numerics.py`, lines 400–780. The Lambert dependency is in `mathics/builtin/specialfns/expintegral.py`. These exact-version observations must be rechecked before attributing them to a later interpreter release.

References

- [1] Vladimir Reshetnikov, *Asymptotic repository*, reviewed commit 7d1bc832895c. [Pinned repository tree](#).
- [2] `src/Kernel/AsymptoticAnalysis.wl`. [Pinned source](#).
- [3] `src/Kernel/NumericalInverseChecks.wl`. [Pinned source](#).
- [4] `src/Kernel/LambertInverse.wl`. [Pinned source](#).
- [5] `src/Kernel/CorePerturbation.wl`. [Pinned source](#).
- [6] `src/Kernel/FourierCoefficients.wl`. [Pinned source](#).
- [7] `src/Kernel/InverseFunctionBranches.wl`. [Pinned source](#).
- [8] `docs/development/CODE_REVIEW_STATUS.md`, native compatibility B01–B04 and implementation status. [Pinned register](#).
- [9] `docs/Mathics/COMPATIBILITY.md`, including validation and preservation receipts. [Pinned document](#).
- [10] `src/Kernel/MathicsCompatibility.wl`. [Pinned source](#).
- [11] `src/Kernel/MathicsCalls.wl`. [Pinned source](#).
- [12] `src/Kernel/MathicsAssumptions.wl`. [Pinned source](#).
- [13] `src/Kernel/MathicsSimplification.wl`. [Pinned source](#).
- [14] `src/Kernel/MathicsTaylor.wl`. [Pinned source](#).
- [15] `src/Kernel/MathicsCalculus.wl`. [Pinned source](#).
- [16] `src/Kernel/MathicsCoreFunctions.wl`. [Pinned source](#).
- [17] `src/Kernel/MathicsInverseBranches.wl`. [Pinned source](#).
- [18] `src/Kernel/MathicsSpecialFunctions.wl`. [Pinned source](#).
- [19] `src/Kernel/MathicsLists.wl`. [Pinned source](#).
- [20] `src/Kernel/MathicsAlgebra.wl`. [Pinned source](#).
- [21] `src/Kernel/MathicsRefinement.wl`. [Pinned source](#).
- [22] `validation/run_mathics_tests.py`. [Pinned source](#).
- [23] `validation/MathicsTests.wl`. [Pinned suite](#).
- [24] `.github/workflows/mathics.yml`. [Pinned workflow configuration](#).
- [25] `external-reports/code-review/README.md`. [Pinned review catalog](#).
- [26] `docs/development/CODE_REVIEW_STATUS.md`. [Pinned issue inventory and complete wave-1/2 crosswalk](#).
- [27] `docs/development/WAVE_3_INTAKE.md`. [Pinned wave-3 crosswalk](#).
- [28] `external-reports/code-review/wave-1/code-review-6/evidence/findings.csv`. [Pinned original ledger](#).
- [29] Mathics3, release tag 10.0.1, `mathics/builtin/numbers/calculus.py`, especially `_BaseFinder`. [Versioned source](#).

- [30] Mathics3, release tag 10.0.1, `mathics/eval/numbers/calculus/optimizers.py`, especially `find_root_newton`. [Versioned source](#).
- [31] Mathics3, release tag 10.0.1, `mathics/eval/nevaluator.py`, `eval_N` and numerical rounding. [Versioned source](#).
- [32] Mathics3, release tag 10.0.1, `mathics/core/number.py`, `get_precision`. [Versioned source](#).
- [33] Mathics3, release tag 10.0.1, `mathics/core/atoms/numerics.py`, `PrecisionReal.round`. [Versioned source](#).
- [34] Mathics3, release tag 10.0.1, `mathics/builtin/specialfns/expintegral.py`, `ProductLog`. [Versioned source](#).
- [35] Wolfram Research, *SeriesData*, current official language reference. <https://reference.wolfram.com/language/ref/SeriesData.html>.
- [36] Wolfram Research, *InverseSeries*, current official language reference. <https://reference.wolfram.com/language/ref/InverseSeries.html>.
- [37] Wolfram Research, *Asymptotic*, current official language reference. <https://reference.wolfram.com/language/ref/Asymptotic.html>.
- [38] Wolfram Research, *AsymptoticSolve*, current official language reference. <https://reference.wolfram.com/language/ref/AsymptoticSolve.html>.
- [39] Wolfram Research, *Limit*, current official language reference; direction defaults and one-sided conventions. <https://reference.wolfram.com/language/ref/Limit.html>.
- [40] Wolfram Research, *FirstPosition*, current official language reference; default, levels and `Heads`. <https://reference.wolfram.com/language/ref/FirstPosition.html>.
- [41] Wolfram Research, *Position*, current official language reference; count-limited form. <https://reference.wolfram.com/language/ref/Position.html>.
- [42] mpmath developers, *Powers and logarithms*, Lambert W function, argument order and branches. <https://mpmath.org/doc/current/functions/powers.html>. Numerical countermodel executed with mpmath 1.3.0.